



# BITS Pilani

Pilani | Dubai | Goa | Hyderabad



Embedded System Design  
Dr. Meetha V Shenoy,  
Dept of EEE, BITS Pilani Pilani Campus

# PL resources –Zedboard Artix-7 FPGA technology

Meetha V Shenoy

# DSP Slices

| Feature      | Value       |
|--------------|-------------|
| -----        | -----       |
| Architecture | DSP48E1     |
| Count        | 220         |
| Multiplier   | 25 x 18 bit |
| Accumulator  | 48 bit      |

| Operations         |                           |
|--------------------|---------------------------|
| -----              |                           |
| $A \times B$       | Multiply                  |
| $(A \times B) + C$ | Multiply-Accumulate (MAC) |
| $(A + B) \times C$ | Pre-add then multiply     |
| $A + B$            | Fast Adder/Accumulator    |
| $A - B$            | Subtractor                |
|                    |                           |

Supports pipelining– will be discussed later.

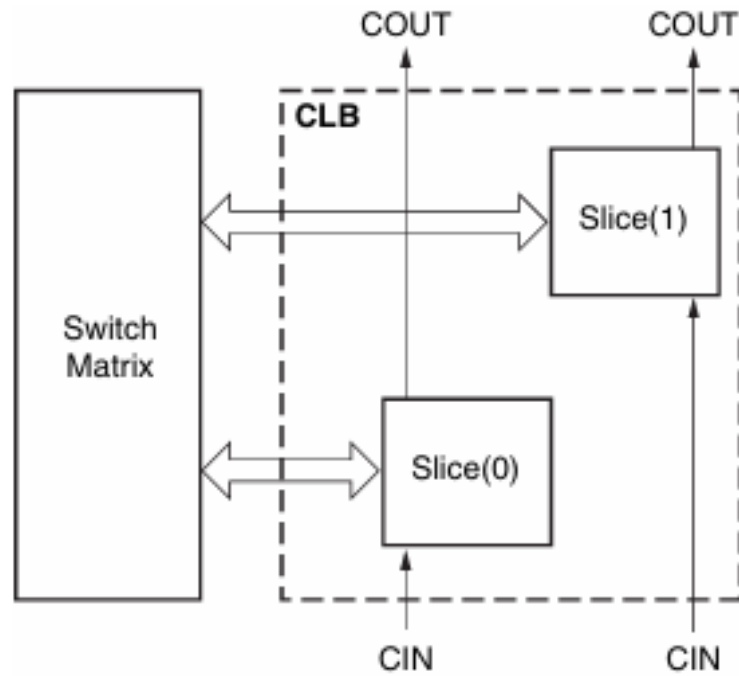
# Block RAM

- It is used heavily for **buffers, FIFOs, local data storage, small memories**
- 36kB
- 140 Number
- Total size -  $140 \times 36 \text{ Kb} = 5040 \text{ Kb} \approx 630 \text{ KB}$

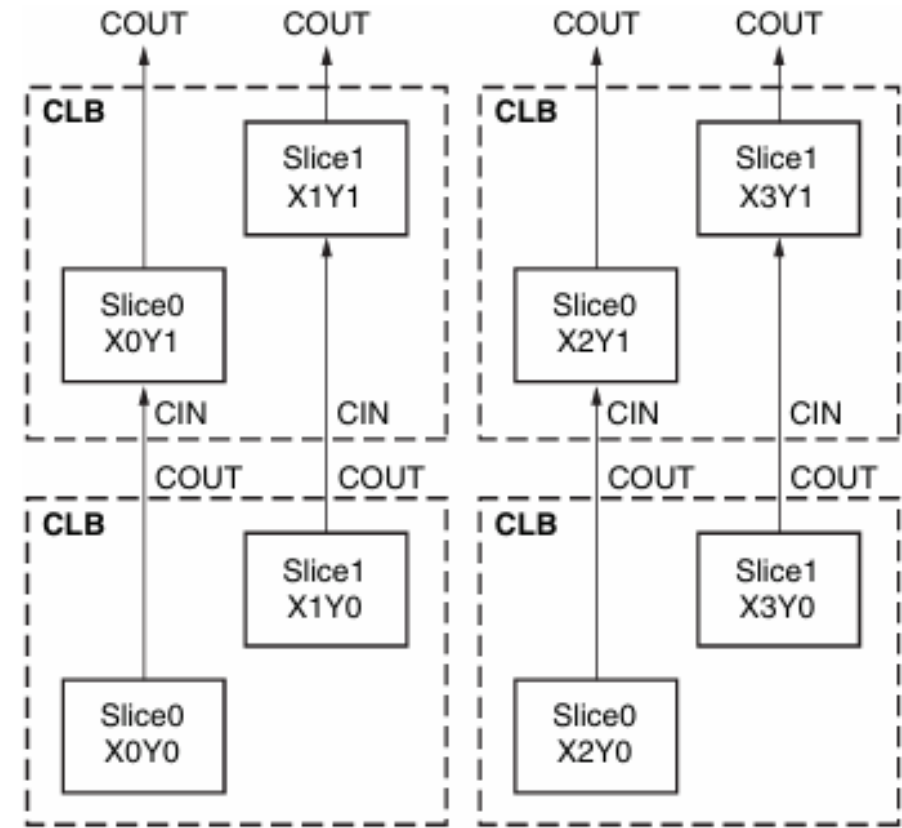
# Configurable Logic Block (CLB)

- A CLB is the fundamental component of an FPGA, allowing the user to implement logical functionality (Sequential or combinational) within the chip.
- In 7-Series FPGA, CLB will have 2 slices.
- Slices are SLICEM, SLICEL
- Each CLB can contain either a SLICEM and SLICEL or two SLICELs.
- These slices contain four look-up tables (LUTs), eight flip-flops (FF), a network of carry logic, and multiplexers .

# CLB



**Arrangement of Slices within the CLB**



**Row and Column Relationship between CLBs and Slices**

# Every Slice contains

- Every slice contains:
  - Four logic-function generators (or look-up tables)
  - Eight storage elements
  - Wide-function Multiplexers
  - Carry logic
- In addition, some slices support two additional functions: storing data using distributed RAM and shifting data with 32-bit registers.
- Slices that support these additional functions are called SLICEM; others are called SLICEL.

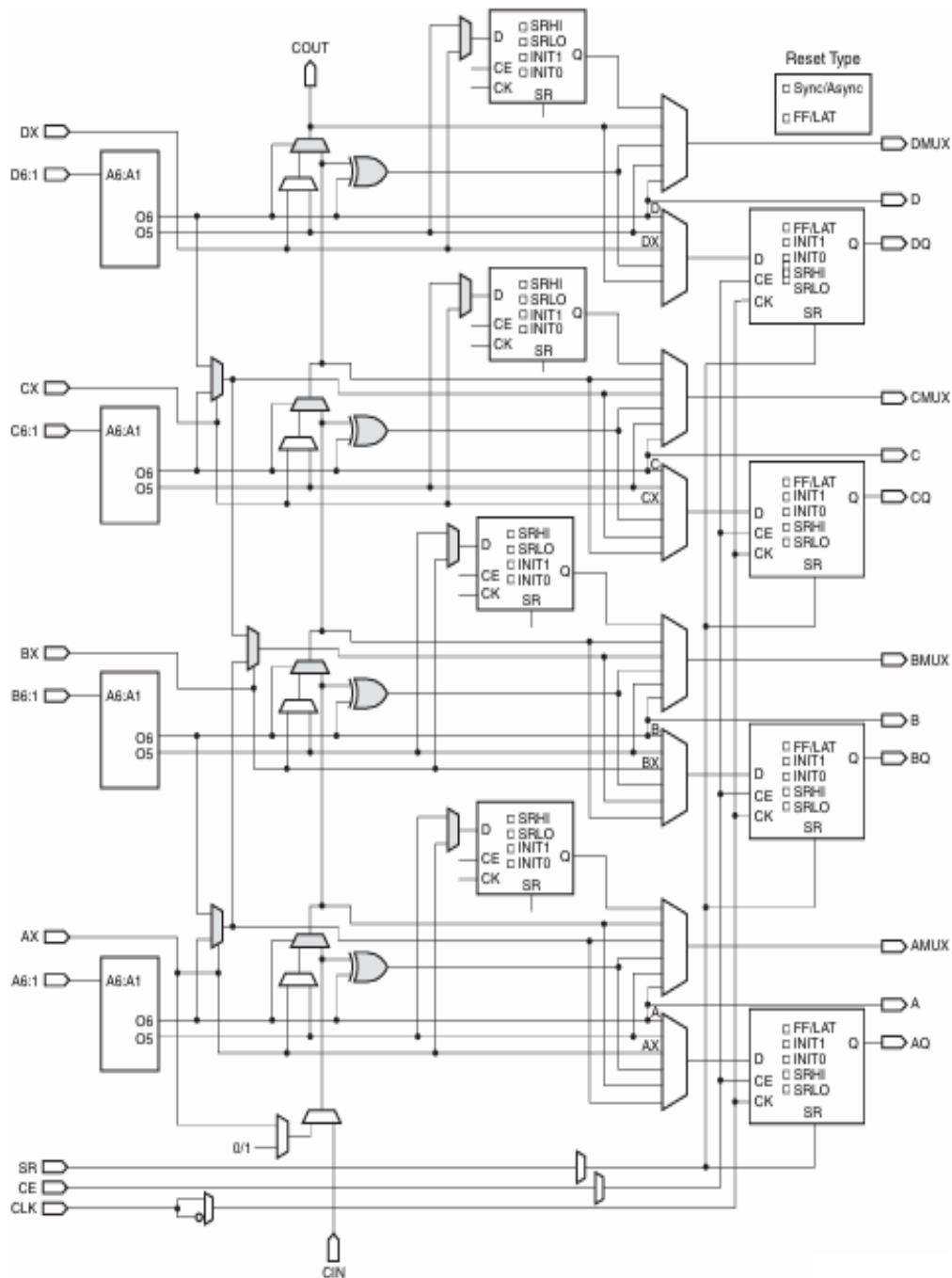
# Look-Up Table (LUT)

- Slices contain four **look-up tables (LUTs)**, eight flip-flops (FF), a network of carry logic, and multiplexers.
- A six-input function uses:
  - A1-A6 inputs
  - O6 output
- Two five-input ( fewer inputs) are used:
  - A1–A5 inputs
  - A6 driven High
  - O5 and O6 outputs

|                                      |
|--------------------------------------|
| O6 exits slice<br>O5 used internally |
|--------------------------------------|

# Eight storage elements

- Each output of the LUT is fed into a storage element
- 8 FF
- 4 latches, 4 unused.



- Input enters an individual LUT (A6:A1, B6:B1, C6:C1, D6:D1). Each LUT can be used independently (6-input), or paired to function as dual 5-input LUTs.
- The output of each LUT (A, B, C, D) is routed directly to either combinational output or sequential logic (flip-flop) via AMUX, BMUX, CMUX, DMUX.
- The primary outputs of a slice are labeled A, B, C, D (combinational outputs) and AQ, BQ, CQ, DQ (registered outputs). These are the signals accessible from outside the slice (i.e., to the FPGA routing fabric).

AX/BX/CX/DX inputs come from the FPGA's general interconnect/routing matrix, just like the regular LUT inputs (A1–A6, B1–B6, etc.). They are independent from the LUT logic signals.

Each of AX, BX, CX, DX is directly connected to one of the four storage elements (flip-flops/latches) in the slice.

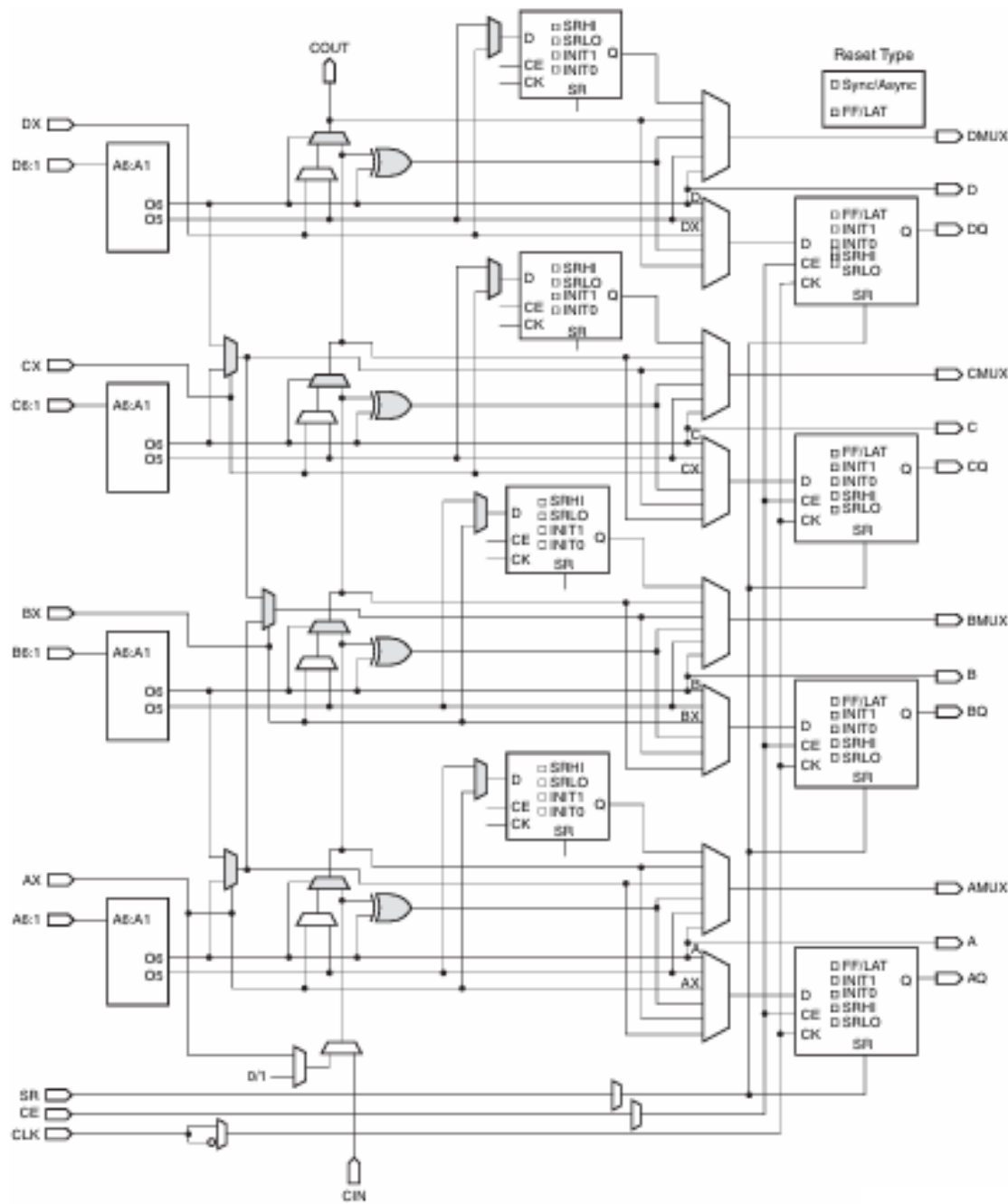
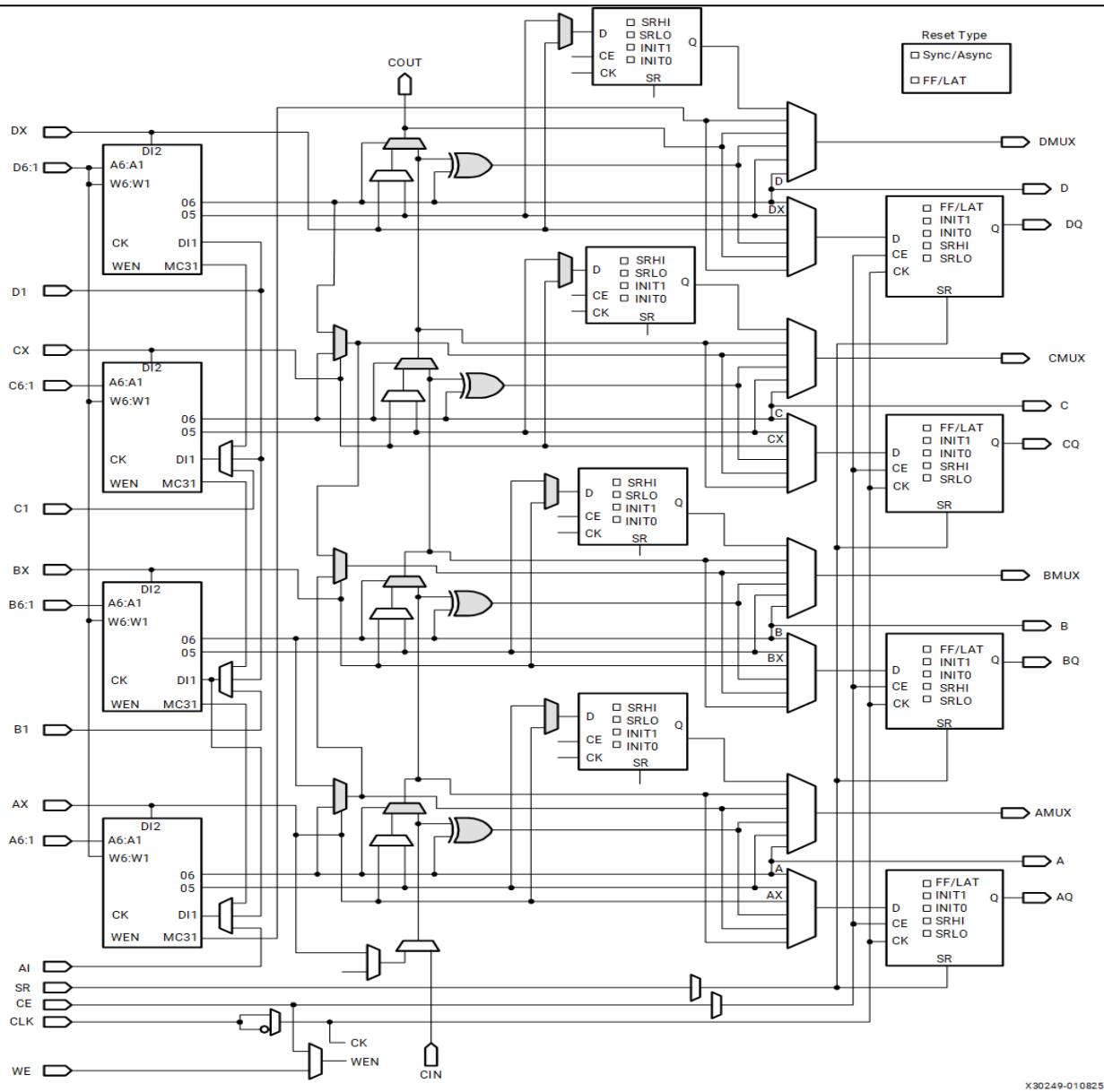
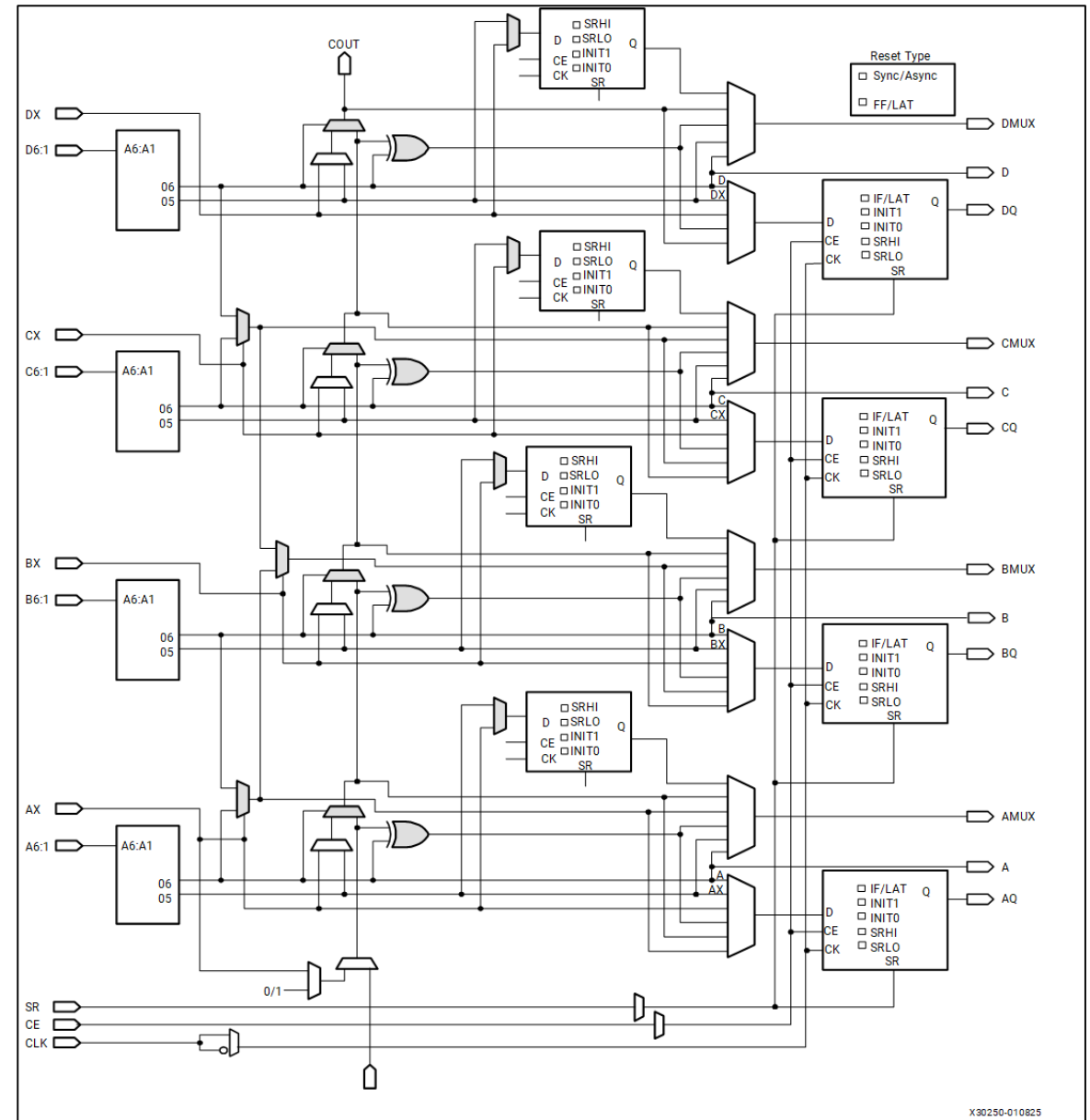


Diagram of SLICEL

# SLICEM



# SLICEL



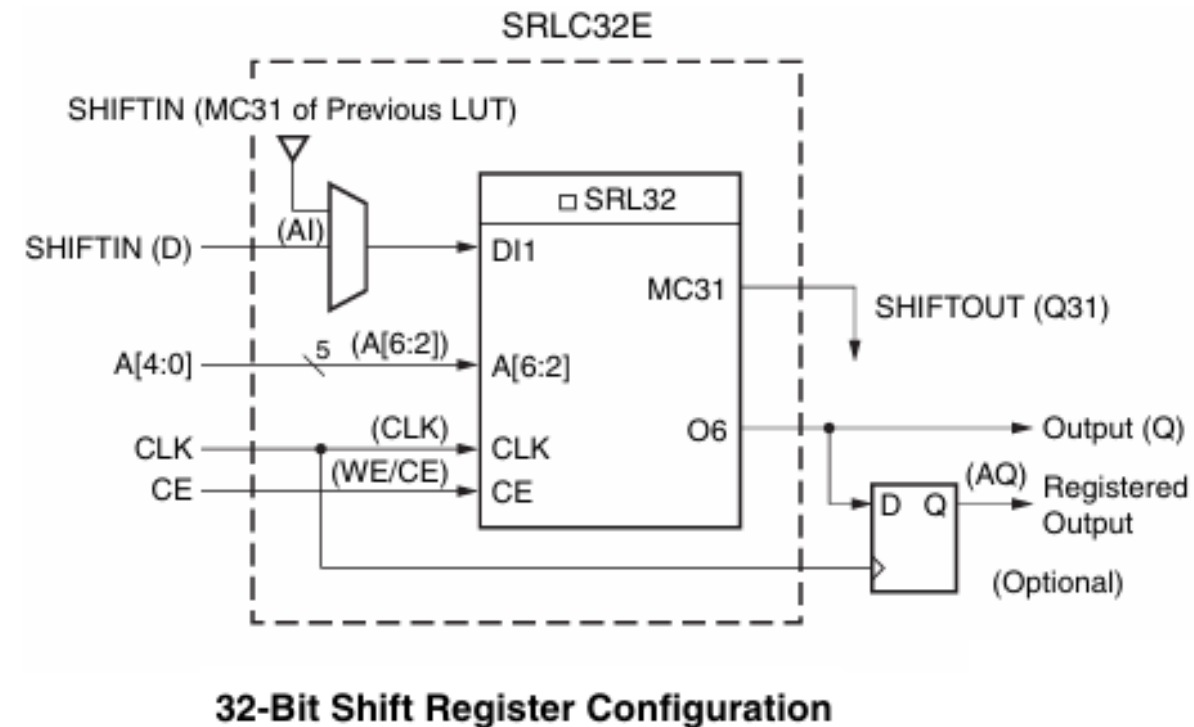
# Shift Register LUT (SRL).

- An **SRL** turns a LUT into a **serial-in / serial-out (SISO)** shift register of configurable depth — no explicit flip-flops required.
- Each LUT6 can act as:
  - an **SRLC32E** (32-bit deep shift register when cascaded with the F7 multiplexer).
- So, instead of using 32 discrete flip-flops for a delay line, you can pack the entire chain into a single LUT — **massive savings** in area and routing.

# 32-bit shift register configuration

set the address (A[4:0])  
input to N-1 to get a  
delay of N cycles.

- Each clock cycle:
- The data input D is written into the first cell.
- Every existing stored bit shifts forward to the next position.
- The bit at the selected position (defined by A[4:0]) appears at Q.
- The oldest bit (after 32 shifts) appears at Q31.
- This behavior is synchronous to CLK and gated by the enable CE.



# Example

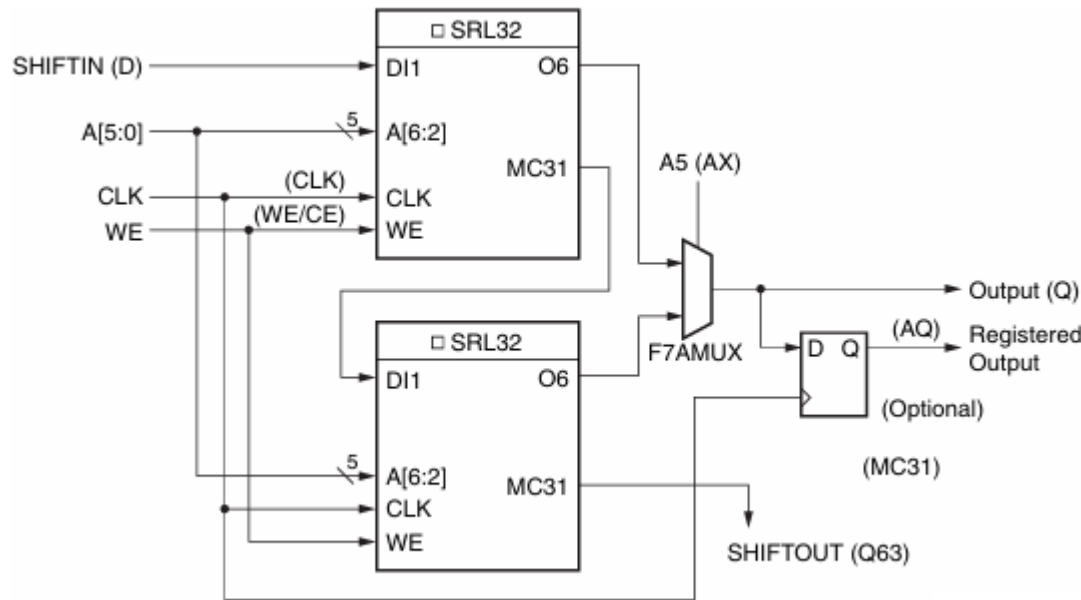
```
SRLC32E delay_inst (  
    .Q(data_out),  
    .Q31(),  
    .A(5'd15),    // delay = 16 cycles  
    .CE(1'b1),  
    .CLK(clk),  
    .D(data_in)  
);
```

Now data\_out is data\_in delayed by 16 clock cycles.

# Shift Register LUT (SRL).

- A SLICEM function generator can also be configured as a 32-bit shift register without using the flip-flops available in a slice.
- Used in this way, each LUT can delay serial data from 1 to 32 clock cycles.
- The shiftin D (DI1 LUT pin) and shiftout Q31 (MC31 LUT pin) lines cascade LUTs to form larger shift registers.

# 64-bit shift register configuration

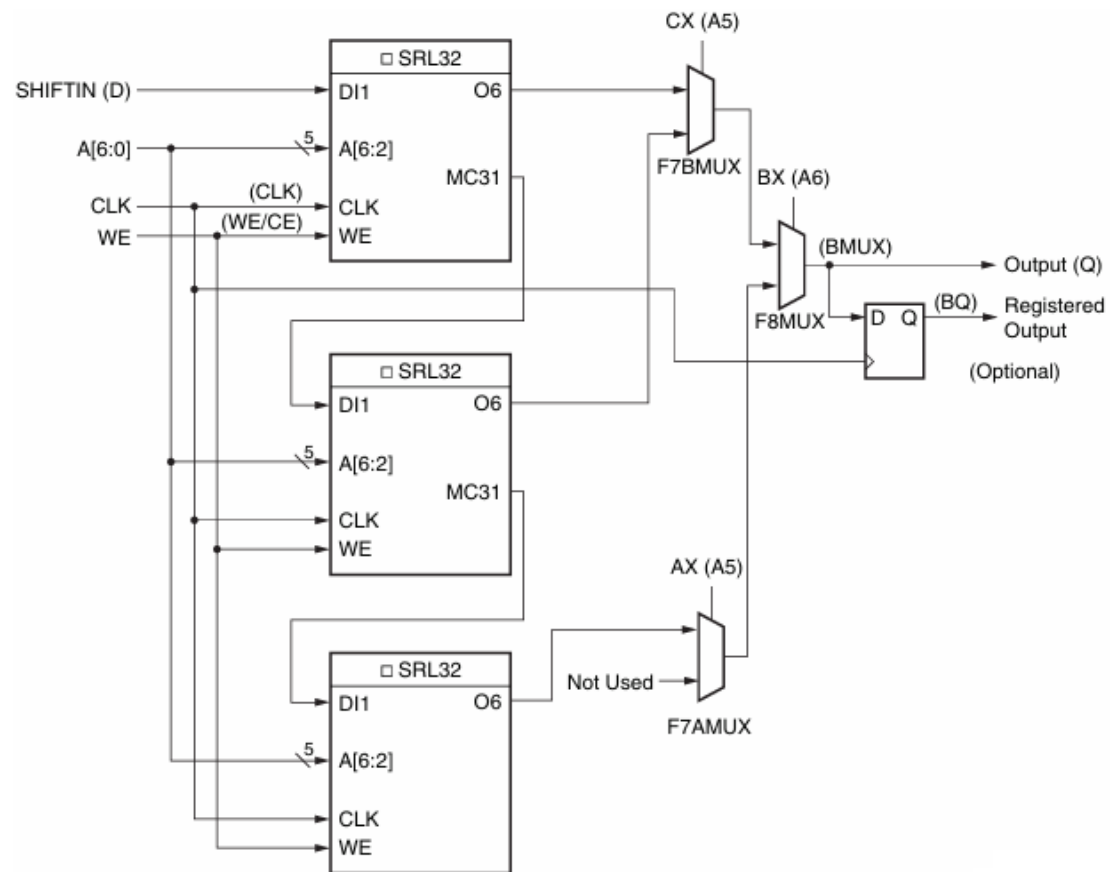


**A[5] becomes the F7MUX select signal.**

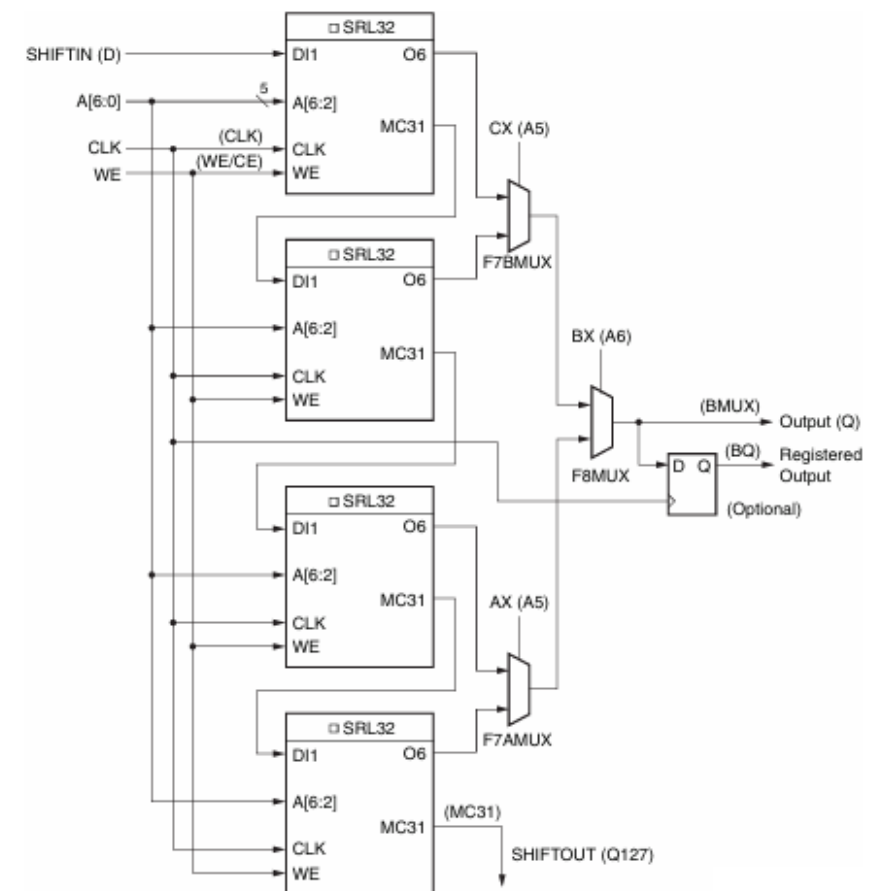
When A5 = 0, SRL1.Q

When A5 = 1, SRL2.Q

| Case         | SRL1.A4..A0             | SRL2.A4..A0 | Q Output Source |
|--------------|-------------------------|-------------|-----------------|
| 1–32 cycles  | (N–1)                   | —           | SRL1.Q          |
| 33–64 cycles | 31 (fixed, always full) | (N–33)      | SRL2.Q          |



### 96-Bit Shift Register Configuration



### 128-Bit Shift Register Configuration

| Range  | SRL Used | A[4:0] (tap) | A5 (F7MUX) | A6 (F8MUX) |
|--------|----------|--------------|------------|------------|
| 1–32   | SRL1     | N–1          | 0          | 0          |
| 33–64  | SRL2     | N–33         | 1          | 0          |
| 65–96  | SRL3     | N–65         | 0          | 1          |
| 97–128 | SRL4     | N–97         | 1          | 1          |

# Shift Register LUT (SRL).

- A SLICEM function generator can also be configured as a 32-bit shift register without using the flip-flops available in a slice.
- Used in this way, each LUT can delay serial data from 1 to 32 clock cycles.
- The shiftin D (DI1 LUT pin) and shiftout Q31 (MC31 LUT pin) lines cascade LUTs to form larger shift registers.
- The four LUTs in a SLICEM are thus cascaded to produce delays up to 128 clock cycles.
- It is also possible to combine shift registers across more than one SLICEM. There are no direct connections between slices to form longer shift registers.

# Distributed Vs Block RAM

| Characteristic | Distributed RAM (LUTs in SLICEM) | Block RAM (BRAM/FIFO36/18)              |
|----------------|----------------------------------|-----------------------------------------|
| Depth          | $\leq 64$ or $128$               | $\geq 512$ (typically)                  |
| Width          | $\leq 8$ bits                    | $\leq 72$ bits (depending on BRAM type) |
| Read type      | Asynchronous                     | Synchronous only                        |
| Write type     | Single port or dual port         | Single/dual port                        |
| Location       | Fabric (uses LUTs)               | Dedicated BRAM tiles                    |

# Map to distributed RAM

```
reg [7:0] mem [0:63]; // 64 x 8 RAM
```

```
(* ram_style = "distributed" *)
```

```
always @(posedge clk) begin  
    if (we)  
        mem[addr] <= din; // write  
end
```

```
assign dout = mem[addr]; // asynchronous read
```

# Block RAM

```
(* ram_style = "block" *) reg [7:0] mem [0:1023]; // 1K x 8 Block  
RAM
```

```
always @(posedge clk) begin  
    if (we)  
        mem[addr] <= din; // synchronous write  
        dout <= mem[addr]; // synchronous read  
end
```

# Pipelining

- Pipelining means **breaking a long combinational path** into **multiple shorter stages**, separated by **flip-flops (registers)**.
- So, instead of one long operation per clock, you have multiple smaller operations happening in parallel - each on a different piece of data.
- Pipelining helps:
  - Achieve **timing closure**
  - Increase **throughput**
  - Optimize for **speed–area trade-offs**

# Pipelining

- Pipelining means **breaking a long combinational path** into **multiple shorter stages**, separated by **flip-flops (registers)**.
- So, instead of one long operation per clock, you have multiple smaller operations happening in parallel - each on a different piece of data.
- Pipelining helps:
  - Achieve **timing closure**
  - Increase **throughput**
  - Optimize for **speed–area trade-offs**

### Non-pipelined

```
always @(posedge clk) begin
    result <= (a + b) * c;
end
```

### Pipelined

```
reg [15:0] add_res;
always @(posedge clk) begin
    add_res <= a + b;    // Stage 1
    result <= add_res * c; // Stage 2
end
```